Podstawy Baz danych

Dzień i godzina zajęć: **środa 13:15**

Nr zespołu: **7**

Autorzy:

- **Patrycja Zborowska**
- **Alicja Czeleń**
- **Piotr Sączawa**

---

1. Wymagania i funkcje systemu

Zarządzanie produktami

- Rejestracja produktów i ich kategorii.
- Definicja struktury produktu (lista części i robocizny).
- Obliczanie kosztu i mocy przerobowej.

Zarządzanie magazynem

- Rejestracja części i stanów magazynowych.
- Rejestracja stanów magazynowych produktów gotowych.
- Aktualizacja stanów magazynowych przy sprzedaży i produkcji.
- Rejestracja planów produkcji.

Planowanie produkcji

- Ustalanie zapotrzebowania na podstawie zamówień.
- Sprawdzanie dostępności surowców.
- Szacowanie czasu produkcji w oparciu o moce przerobowe.
- Tworzenie planu produkcji produktów brakujących w magazynie.

Zarządzanie zamówieniami

- Składanie zamówień przez klientów indywidualnych i firmowych.
- Obsługa płatności.
- Możliwość dodania jednostkowego rabatu.
- Sprawdzenie dostępności produktów w magazynie.
- Obsługa zamówień wymagających produkcji.

Raportowanie (analityka danych)

- Raporty kosztów produkcji: jednostkowe, grup produktowych; kwartalne/miesięczne/roczne.
- Raporty stanów magazynowych (produkty i surowce) oraz planów produkcji.
- Raporty zamówień klientów (także z rabatami) z opcją filtrowania po okresach czasu.

- Raporty sprzedaży grup produktów (miesiące i tygodnie).
- Raport kosztów produkcji w ujęciu tygodniowym i miesięcznym.
- Raporty planów wytwórczych w wybranych okresach.

Narzędzia bazy danych

- Procedury, funkcje (np. obliczanie kosztów).
- Triggery aktualizujące stany.
- Widoki analityczne do raportów.

Use cases:

UC1: Rejestracja produktu

Aktor: Administrator / Planista produkcji **Cel:** Dodanie nowego produktu do systemu. **Wyzwalacz:** Potrzeba wprowadzenia nowego produktu.

Główny przebieg:

- Aktor wybiera opcję „Dodaj produkt".
- Wprowadza dane: nazwę produktu, kategorię, cenę, parametry techniczne, listę części.
- System waliduje dane.
- System zapisuje produkt w bazie.

UC2: Obliczenie kosztu produkcji

Aktor: Planista produkcji **Cel:** Wyliczenie jednostkowego kosztu produktu. **Dane wejściowe:** ID produktu.

Główny przebieg:

- Planista wybiera produkt.
- System pobiera listę części i ich koszty.
- System pobiera dane o czasie pracy.
- System sumuje koszty materiałów i pracy.
- System wyświetla jednostkowy koszt produkcji.

UC3: Sprawdzenie dostępności magazynowej

Aktor: Magazynier / System produkcji **Cel:** Zweryfikowanie dostępności produktu lub jego części. **Dane wejściowe:** ID produktu.

Główny przebieg:

- Aktor lub system inicjuje sprawdzenie stanu magazynowego.
- System pobiera stany magazynowe produktu oraz wymaganych części.
- System określa poziom dostępności.

Wynik:

- „dostępny”
- „brak części”
- „brak produktu”

UC4: Złożenie zamówienia

Aktor: Klient / Dział sprzedaży **Cel:** Zarejestrowanie zamówienia w systemie.

Główny przebieg:

- Klient wybiera produkty.
- System sprawdza dostępność magazynową.
- System nalicza rabaty, jeśli obowiązują.
- System wylicza całkowity koszt zamówienia.
- System zapisuje zamówienie.
- Klient lub dział sprzedaży potwierdza zamówienie.

UC5: Planowanie produkcji

Aktor: Planista produkcji **Cel:** Utworzenie planu produkcji.

Główny przebieg:

- System pobiera zamówienia wymagające produkcji.
- System analizuje dostępność części.
- System określa wymagany czas produkcji.
- Planista zatwierdza plan.

UC6: Realizacja produkcji

Aktor: Pracownik produkcji / System produkcji **Cel:** Zrealizowanie procesu produkcyjnego.

Główny przebieg:

- System generuje zapotrzebowanie na części.
- Magazyn wydaje części i aktualizuje stany magazynowe.
- Produkcja wytwarza produkt.
- System rejestruje produkt gotowy w magazynie.

UC7: Generowanie raportów

Aktor: Zarząd / Analityk **Cel:** Analiza danych produkcyjnych, magazynowych i sprzedażowych.

Rodzaje raportów:

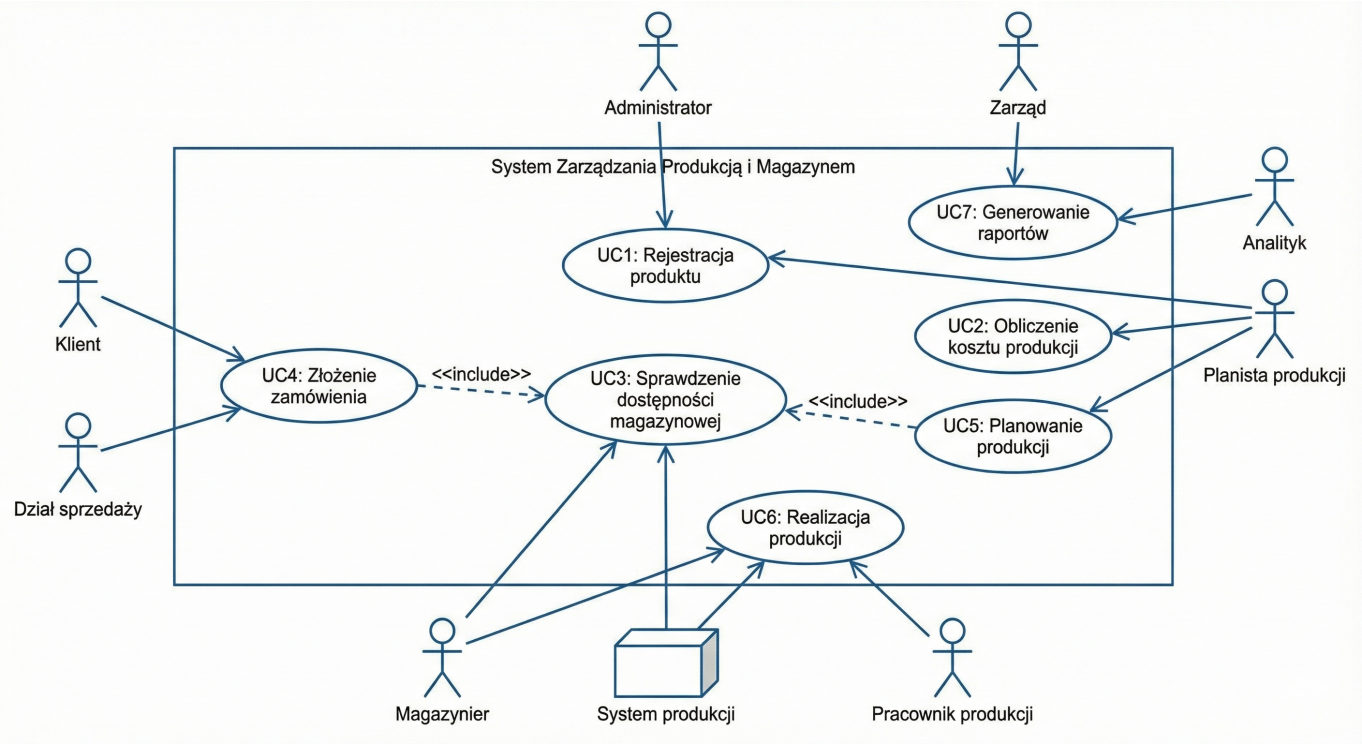
- koszty produkcji (wg okresów),

- stany magazynowe,
- zamówienia klientów,
- sprzedaż tygodniowa/miesięczna,
- plany i realizacja produkcji.

Główny przebieg:

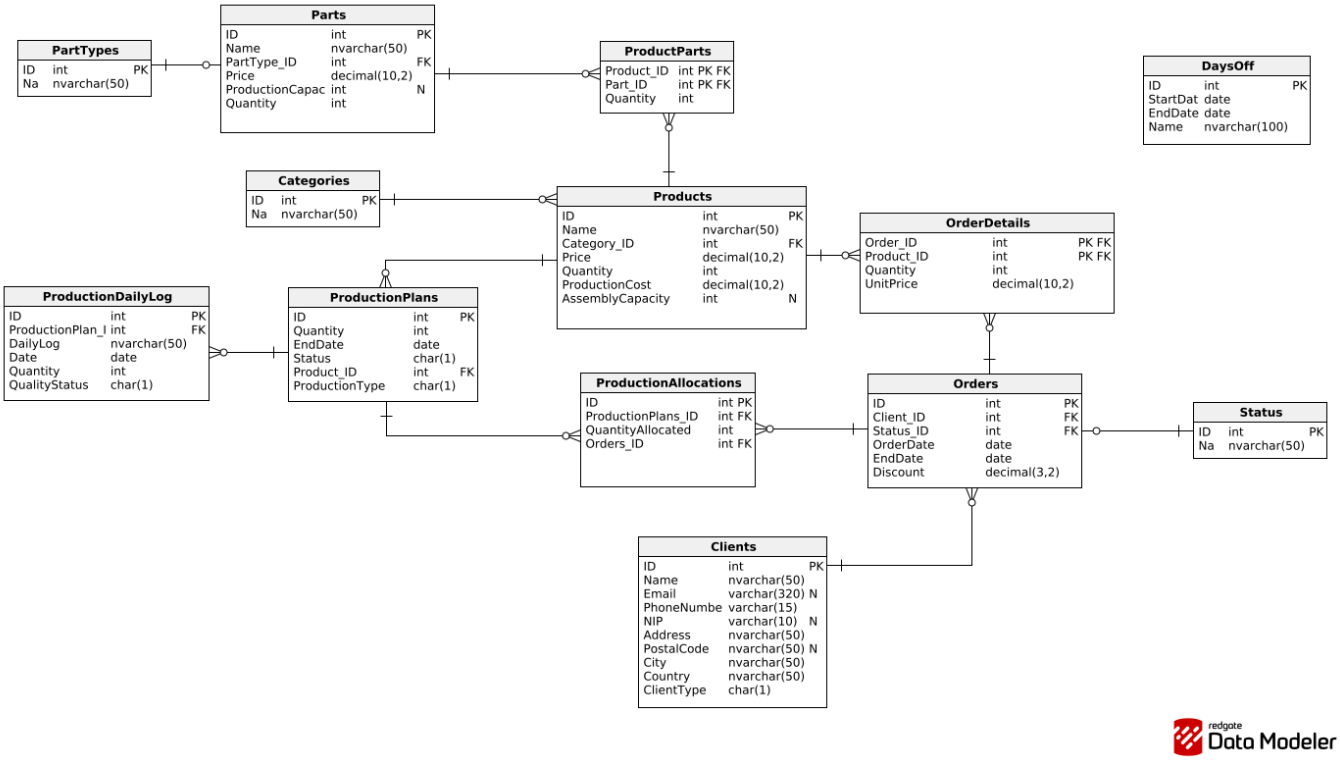
- Aktor wybiera typ raportu oraz zakres dat.
- System pobiera wymagane dane.
- System generuje raport i udostępnia go do wglądu lub eksportu.

Diagram przypadków użycia



2. Baza danych

Schemat bazy danych



Opis poszczególnych tabel

Kategorie produktów

```
CREATE TABLE dbo.Categories (
    ID int  NOT NULL IDENTITY,
    Name nvarchar(50)  NOT NULL,
    CONSTRAINT PK_Categories PRIMARY KEY CLUSTERED (ID)
);
```

Klienci

```
CREATE TABLE dbo.Clients (
    ID int  NOT NULL IDENTITY,
    Name nvarchar(50)  NOT NULL,
    Email varchar(320)  NULL,
    PhoneNumber varchar(15)  NOT NULL,
    NIP varchar(10)  NULL,
    Address nvarchar(50)  NOT NULL,
    PostalCode nvarchar(50)  NULL,
    City nvarchar(50)  NOT NULL,
    Country nvarchar(50)  NOT NULL,
    ClientType char(1)  NOT NULL,
    CONSTRAINT PK_Clients PRIMARY KEY CLUSTERED (ID)
);
```

Dni bez pracy/produkcji

```sql
CREATE TABLE DaysOff (
    ID int  NOT NULL IDENTITY,
    StartDate date  NOT NULL,
    EndDate date  NOT NULL,
    Name nvarchar(100)  NOT NULL,
    CONSTRAINT DaysOff_pk PRIMARY KEY  (ID)
);
```

Szczegóły zamówienia

```sql
CREATE TABLE dbo.OrderDetails (
    Order_ID int  NOT NULL,
    Product_ID int  NOT NULL,
    Quantity int  NOT NULL,
    UnitPrice decimal(10,2)  NOT NULL,
    CONSTRAINT PK_OrderDetails PRIMARY KEY CLUSTERED (Order_ID,Product_ID)
);
```

Zamówienia

```sql
CREATE TABLE dbo.Orders (
    ID int  NOT NULL IDENTITY,
    Client_ID int  NOT NULL,
    Status_ID int  NOT NULL,
    OrderDate date  NOT NULL,
    EndDate date  NOT NULL,
    Discount decimal(3,2)  NOT NULL DEFAULT 0,
    CONSTRAINT PK_Orders PRIMARY KEY CLUSTERED (ID)
);
```

Kategorie części

```sql
CREATE TABLE dbo.PartTypes (
    ID int  NOT NULL IDENTITY,
    Name nvarchar(50)  NOT NULL,
    CONSTRAINT PK_PartTypes PRIMARY KEY CLUSTERED (ID)
);
```

Części

```sql
CREATE TABLE dbo.Parts (
    ID int  NOT NULL IDENTITY,
    Name nvarchar(50)  NOT NULL,
```

```
    PartType_ID int  NOT NULL,
    Price decimal(10,2)  NOT NULL,
    ProductionCapacity int  NULL,
    Quantity int  NOT NULL DEFAULT 0,
    CONSTRAINT PK_Parts PRIMARY KEY CLUSTERED (ID)
);
```

Części danego produktu (łącznikowa)

```
CREATE TABLE dbo.ProductParts (
    Product_ID int  NOT NULL,
    Part_ID int  NOT NULL,
    Quantity int  NOT NULL,
    CONSTRAINT PK_ProductParts PRIMARY KEY CLUSTERED (Product_ID,Part_ID)
);
```

Zarezerwowanie produkowanych rzeczy do konkretnego zamówienia

```
CREATE TABLE ProductionAllocations (
    ID int  NOT NULL IDENTITY,
    ProductionPlans_ID int  NOT NULL,
    QuantityAllocated int  NOT NULL,
    Orders_ID int  NOT NULL,
    CONSTRAINT ProductionAllocations_pk PRIMARY KEY  (ID)
);
```

Dzienne sprawozdanie z wykonywania planu produkcyjnego

```
CREATE TABLE ProductionDailyLog (
    ID int  NOT NULL IDENTITY,
    ProductionPlan_ID int  NOT NULL,
    DailyLog nvarchar(50)  NOT NULL,
    Date date  NOT NULL,
    Quantity int  NOT NULL,
    QualityStatus char(1)  NOT NULL,
    CONSTRAINT ProductionDailyLog_pk PRIMARY KEY  (ID)
);
```

Plany produkcyjne (cykliczne bądź wymuszone popytem)

```
CREATE TABLE dbo.ProductionPlans (
    ID int  NOT NULL IDENTITY,
    Quantity int  NOT NULL,
    EndDate date  NOT NULL,
```

```
    Status char(1)  NOT NULL,
    Product_ID int  NOT NULL,
    ProductionType char(1)  NOT NULL,
    CONSTRAINT PK_ProductionPlans PRIMARY KEY CLUSTERED (ID)
);
```

Produkty

```
CREATE TABLE dbo.Products (
    ID int NOT NULL IDENTITY,
    Name nvarchar(50) NOT NULL,
    Category_ID int NOT NULL,
    Quantity int NOT NULL DEFAULT 0,
    AssemblyCapacity int NULL,

    ProductionCost AS ([dbo].[ObliczKosztProdukcji]([ID])),
    Price AS ([dbo].[ObliczKosztProdukcji]([ID]) * 1.5),

    CONSTRAINT PK_Products PRIMARY KEY CLUSTERED (ID),
    CONSTRAINT FK_Products_Categories FOREIGN KEY (Category_ID) REFERENCES
dbo.Categories(ID)
);
```

- Opis: Kolumny ProductionCost i Price wyliczane funkcją poniżej.

Statusy zamówień

```
CREATE TABLE dbo.Status (
    ID int  NOT NULL IDENTITY,
    Name nvarchar(50)  NOT NULL,
    CONSTRAINT PK_Status PRIMARY KEY CLUSTERED (ID)
);
```

Procedury/funkcje

```
CREATE FUNCTION dbo.ObliczKosztProdukcji (@ProductId INT)
    RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @Koszt DECIMAL(10,2);

    -- Sumujemy (Cena Części * Ilość Części w Produkcie)
    SELECT @Koszt = SUM(p.Price * pp.Quantity)
    FROM Parts p
             JOIN ProductParts pp ON p.ID = pp.Part_ID
    WHERE pp.Product_ID = @ProductId;
```

```
    -- Zwracamy 0 jeśli brak części, w przeciwnym razie wyliczoną sumę
    RETURN ISNULL(@Koszt, 0);
END
```

- Opis: Funkcja obliczająca koszt produkcji danego produktu na podstawie ilości i ceny jego części